AI ASSISTED CODING

Assignment Number: 13.1

Name: Hansika Vangari HallTicket: 2303A52409

Lab 13: Code Refactoring – Improving Legacy Code with AI Suggestions

Lab Objectives:

• Identify code smells and inefficiencies in legacy Python scripts.

• Use AI-assisted coding tools to refactor for readability, maintainability, and performance.

• Apply modern Python best practices while ensuring output correctness.

Task Description #1 (Refactoring – Removing Code Duplication)

• Task: Use AI to refactor a given Python script that contains multiple repeated code blocks.

• Instructions:

o Prompt AI to identify duplicate logic and replace it with functions or classes.

o Ensure the refactored code maintains the same output.

o Add docstrings to all functions.

• Sample Legacy Code:

```
# Legacy script with repeated logic

print("Area of Rectangle:", 5 * 10)

print("Perimeter of Rectangle:", 2 * (5 + 10))
```

```
print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))
print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

• Expected Output:

o Refactored code with a reusable function and no duplication.

o Well documented code

Code:

```python
#identify the duplicate logic and correct the code and replace it with functions or classes
#and refactor the  below code
#add docstrings to the code
def calculate_rectangle_area(length, width):
    """
    Calculate the area of a rectangle.

    Args:
        length (float): The length of the rectangle.
        width (float): The width of the rectangle.

    Returns:
        float: The area of the rectangle.
    """
    return length * width

def calculate_rectangle_perimeter(length, width):
    """
    Calculate the perimeter of a rectangle.

    Args:
        length (float): The length of the rectangle.
        width (float): The width of the rectangle.

    Returns:
        float: The perimeter of the rectangle.
    """
    return 2 * (length + width)

print("Area of Rectangle:", calculate_rectangle_area(5, 10))
print("Perimeter of Rectangle:", calculate_rectangle_perimeter(5, 10))
print("Area of Rectangle:", calculate_rectangle_area(7, 12))
print("Perimeter of Rectangle:", calculate_rectangle_perimeter(7, 12))
print("Area of Rectangle:", calculate_rectangle_area(10, 15))
print("Perimeter of Rectangle:", calculate_rectangle_perimeter(10, 15))
```

Output:

```
PS C:\Users\HP\OneDrive\Documents\AIAC_2026> & C:
uments/AIAC_2026/Lab-11.py
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
```

Task Description #2 (Refactoring – Extracting Reusable Functions)

• Task: Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.

• Instructions:

o Identify repeated or related logic and extract it into reusable functions.

o Ensure the refactored code is modular, easy to read, and documented with docstrings.

• Sample Legacy Code:

```
# Legacy script with inline repeated logic

price = 250

tax = price * 0.18

total = price + tax

print("Total Price:", total)

price = 500

tax = price * 0.18

total = price + tax

print("Total Price:", total)
```

• Expected Output:

o Code with a function calculate_total(price) that can be reused for multiple price inputs.

o Well documented code

Code:

```python
#refactor the above code identify repeated or related logic and extract it into reusable function calculate_total(price) and
# generate a well documented code with docstrings
def calculate_total(price):
    """Calculate the total price including tax.

    Args:
        price (float): The original price of the item.

    Returns:
        float: The total price after adding tax.
    """
    tax = price * 0.18
    total = price + tax
    return total
# Example usage
if __name__ == "__main__":
    price1 = 250
    total1 = calculate_total(price1)
    print("Total Price:", total1)

    price2 = 500
    total2 = calculate_total(price2)
    print("Total Price:", total2)
```

Output:

```
Total Price: 295.0
Total Price: 590.0
Total Price: 295.0
Total Price: 590.0
```

Task Description #3: Refactoring Using Classes and Methods (Eliminating Redundant Conditional Logic)

Refactor a Python script that contains repeated if–elif–else grading logic by implementing a structured, object-oriented solution using a class and a method.

Problem Statement

The given script contains duplicated conditional statements used to assign grades based on student marks. This redundancy violates clean code principles and reduces maintainability.

You are required to refactor the script using a class-based design to improve modularity, reusability, and readability while preserving the original grading logic.

Mandatory Implementation Requirements

1. Class Name: GradeCalculator

2. Method Name: calculate_grade(self, marks)

3. The method must:

o Accept marks as a parameter.

o Return the corresponding grade as a string.

o The grading logic must strictly follow the conditions below:

▪ Marks ≥ 90 and ≤ 100 → "Grade A"

▪ Marks ≥ 80 → "Grade B"

▪ Marks ≥ 70 → "Grade C"

▪ Marks ≥ 40 → "Grade D"

▪ Marks ≥ 0 → "Fail"

Note: Assume marks are within the valid range of 0 to 100.

4. Include proper docstrings for:

o The class

o The method (with parameter and return descriptions)

5. The method must be reusable and called multiple times without rewriting conditional logic.

• Given code:

```
marks = 85
if marks >= 90:
print("Grade A")
elif marks >= 75:
print("Grade B")
else:
print("Grade C")
marks = 72
if marks >= 90:
print("Grade A")
elif marks >= 75:
print("Grade B")
else:
print("Grade C")
```

Expected Output:

• Define a class named GradeCalculator.

• Implement a method calculate_grade(self, marks) inside the class.

• Create an object of the class.

• Call the method for different student marks.

• Print the returned grade values.

Code:

```python
#refactor the above code with a class-GradeCalculatorand invoke a method calculate_grade(self,marks)
#  to determine the grade based on the marks and generate a well documented code with docstrings
class GradeCalculator:
    def calculate_grade(self, marks):
        """Determine the grade based on the marks.

        Args:
            marks (int): The marks obtained by the student.

        Returns:
            str: The grade corresponding to the marks.
        """
        if marks >= 90 and marks <= 100:
            return "Grade A"
        elif marks >= 80:
            return "Grade B"
        elif marks >= 70:
            return "Grade C"
        elif marks >= 40:
            return "Grade D"
        elif marks >= 0:
            return "Grade F"
        else:
            return "Invalid marks"
# Example usage
if __name__ == "__main__":
    grade_calculator = GradeCalculator()

    marks1 = 85
    grade1 = grade_calculator.calculate_grade(marks1)
    print(f"Marks: {marks1}, {grade1}")

    marks2 = 72
    grade2 = grade_calculator.calculate_grade(marks2)
    print(f"Marks: {marks2}, {grade2}")
```

Output:

```
Marks: 85, Grade B
Marks: 72, Grade C
```

Task Description #4 (Refactoring – Converting Procedural Code to Functions)

• Task: Use AI to refactor procedural input–processing logic into functions.

Instructions:

o Identify input, processing, and output sections.

o Convert each into a separate function.

o Improve code readability without changing behavior.

• Sample Legacy Code:

```
num = int(input("Enter number: "))

square = num * num

print("Square:", square)
```

• Expected Output:

o Modular code using functions like get_input(), calculate_square(), and display_result().

Code:

```python
#refactor the above code using seperate function with docstrings
def calculate_square(num):
    """Calculate the square of a number.

    Args:
        num (int): The number to be squared.

    Returns:
        int: The square of the input number.
    """
    return num * num
# Example usage
if __name__ == "__main__":
    num = int(input("Enter number: "))
    square = calculate_square(num)
    print("Square:", square)
```

Output:

```
Enter number: 5
Square: 25
```

Task 5 (Refactoring Procedural Code into OOP Design)

• Task: Use AI to refactor procedural code into a class-based design.

Focus Areas:

o Object-Oriented principles

o Encapsulation

Legacy Code:

salary = 50000

tax = salary * 0.2

net = salary - tax

print(net)

Expected Outcome:

o A class like EmployeeSalaryCalculator with methods and attributes.

Code:

```
#refactor the above code with a class-EmployeeSalaryCalculator
# with methods and attributes and invoke object-oriented principle
#  Encapsulation
class EmployeeSalaryCalculator:
    def __init__(self, salary):
        self.salary = salary

    def calculate_tax(self):
        """Calculate the tax based on the salary."""
        return self.salary * 0.2

    def calculate_net_salary(self):
        """Calculate the net salary after deducting tax."""
        tax = self.calculate_tax()
        net_salary = self.salary - tax
        return net_salary
# Example usage
if __name__ == "__main__":
    employee_salary_calculator = EmployeeSalaryCalculator(50000)
    net_salary = employee_salary_calculator.calculate_net_salary()
    print("Net Salary:", net_salary)
```

Output:

```
Net Salary: 40000.0
```

Task 6 (Optimizing Search Logic)

• Task: Refactor inefficient linear searches using appropriate data structures.

• Focus Areas:

o Time complexity

o Data structure choice

Legacy Code:

users = ["admin", "guest", "editor", "viewer"]

name = input("Enter username: ")

found = False

for u in users:

if u == name:

found = True

print("Access Granted" if found else "Access Denied")

Expected Outcome:

o Use of sets or dictionaries with complexity justification

Code:

```
#refactor the above code  using sets or dictoinaries  with optimized time complexity
users = {"admin", "guest", "editor", "viewer"}  # Using a set for O(1) average time complexity
name = input("Enter username: ")
if name in users:
    print("Access Granted")
else:
    print("Access Denied")
```

Output:

```
Enter username: john
Access Denied
```

Task 7 – Refactoring the Library Management System

Problem Statement

You are provided with a poorly structured Library Management script that:

• Contains repeated conditional logic

• Does not use reusable functions

• Lacks documentation

• Uses print-based procedural execution

• Does not follow modular programming principles

Your task is to refactor the code into a proper format

1. Create a module library.py with functions:

o add_book(title, author, isbn)

o remove_book(isbn)

o search_book(isbn)

2. Insert triple quotes under each function and let Copilot complete the docstrings.

3. Generate documentation in the terminal.

4. Export the documentation in HTML format.

5. Open the file in a browser.

Given Code

```
# Library Management System (Unstructured Version)

# This code needs refactoring into a proper module with documentation.

library_db = {}

# Adding first book

title = "Python Basics"

author = "John Doe"

isbn = "101"

if isbn not in library_db:

library_db[isbn] = {"title": title, "author": author}

print("Book added successfully.")

else:

print("Book already exists.")

# Adding second book (duplicate logic)

title = "AI Fundamentals"

author = "Jane Smith"

isbn = "102"

if isbn not in library_db:
```

```
library_db[isbn] = {"title": title, "author": author}
print("Book added successfully.")
else:
print("Book already exists.")
# Searching book (repeated logic structure)
isbn = "101"
if isbn in library_db:
print("Book Found:", library_db[isbn])
else:
print("Book not found.")
# Removing book (again repeated pattern)
isbn = "101"
if isbn in library_db:
del library_db[isbn]
print("Book removed successfully.")
else:
print("Book not found.")
# Searching again
isbn = "101"
if isbn in library_db:
```

print("Book Found:", library_db[isbn])

else:

print("Book not found.")

Code:

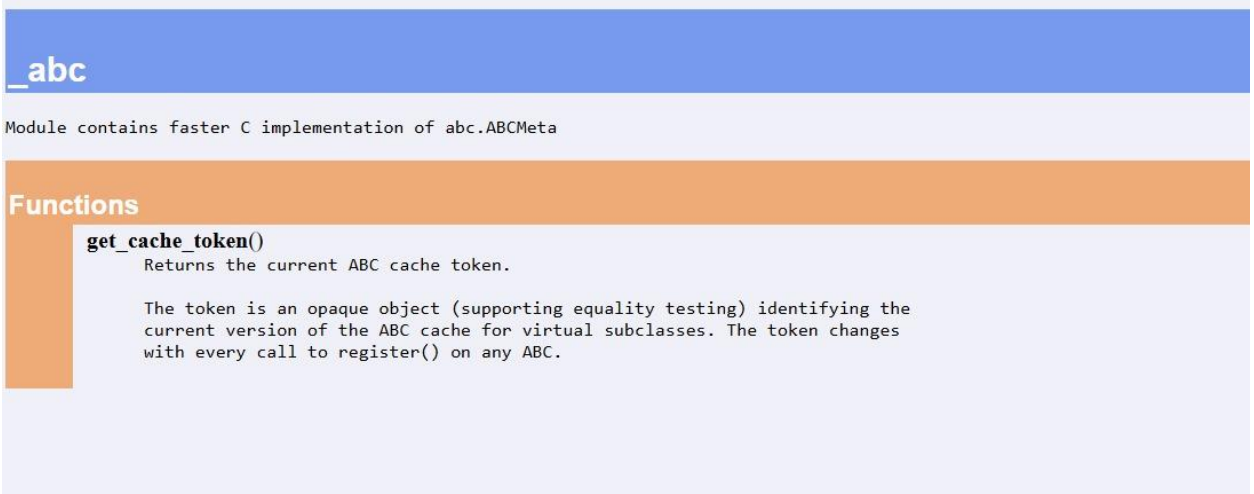
```
Welcome   dsawithai.py   aiac13.py   library.py
C: > Users > Admin > Downloads > AI ASSISTENCE > library.py > ...
#refactor the above code with add_book(title,author,isbn),search_book(isbn) and remove_book(isbn) functions and generate a well documented code with docstrings
library_db = {}
def add_book(title, author, isbn):
    """Add a book to the library database.

    Args:
        title (str): The title of the book.
        author (str): The author of the book.
        isbn (str): The ISBN number of the book.

    Returns:
        str: A message indicating whether the book was added successfully or if it already exists.
    """
    if isbn not in library_db:
        library_db[isbn] = {"title": title, "author": author}
        return "Book added successfully."
    else:
        return "Book already exists."
def search_book(isbn):
    """Search for a book in the library database by its ISBN.

    Args:
        isbn (str): The ISBN number of the book to search for.

    Returns:
        dict or str: The book details if found, or a message indicating that the book was not found.
    """
    if isbn in library_db:
        return library_db[isbn]
    else:
        return "Book not found."
def remove_book(isbn):
    """Remove a book from the library database by its ISBN.

    Args:
        isbn (str): The ISBN number of the book to remove.
```

```
def remove_book(isbn):
    """Remove a book from the library database by its ISBN.

    Args:
        isbn (str): The ISBN number of the book to remove.
    Returns:
        str: A message indicating whether the book was removed successfully or if it was not found.
    """
    if isbn in library_db:
        del library_db[isbn]
        return "Book removed successfully."
    else:
        return "Book not found."
# Example usage
if __name__ == "__main__":
    print(add_book("Python Basics", "John Doe", "101"))
    print(add_book("AI Fundamentals", "Jane Smith", "102"))
    print(search_book("101"))
    print(remove_book("101"))
    print(search_book("101"))
```

Output:

```
PS C:\Users\Admin\Downloads\AI ASSISTENCE> & C:\Users\Admin\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/Admin/Downloads/AI ASSISTENCE/librar
y.py"
Book added successfully.
Book added successfully.
{'title': 'Python Basics', 'author': 'John Doe'}
Book removed successfully.
Book not found.
PS C:\Users\Admin\Downloads\AI ASSISTENCE> python -m pydoc library
Help on module library:

NAME
    library - #refactor the above code with add_book(title,author,isbn),search_book(isbn) and remove_book(isbn) functions and generate a well documented code
 with docstrings

FUNCTIONS
    add_book(title, author, isbn)
        Add a book to the library database.

        Args:
            title (str): The title of the book.
            author (str): The author of the book.
            isbn (str): The ISBN number of the book.

        Returns:
            str: A message indicating whether the book was added successfully or if it already exists.

    remove_book(isbn)
        Remove a book from the library database by its ISBN.

        Args:
            isbn (str): The ISBN number of the book to remove.
        Returns:
            str: A message indicating whether the book was removed successfully or if it was not found.

    search_book(isbn)
        Search for a book in the library database by its ISBN.

        Args:
            isbn (str): The ISBN number of the book to search for.
```

```
            isbn (str): The ISBN number of the book.

        Returns:
            str: A message indicating whether the book was added successfully or if it already exists.

    remove_book(isbn)
        Remove a book from the library database by its ISBN.

        Args:
            isbn (str): The ISBN number of the book to remove.
        Returns:
            str: A message indicating whether the book was removed successfully or if it was not found.

    search_book(isbn)
        Search for a book in the library database by its ISBN.

        Args:
            isbn (str): The ISBN number of the book to search for.

        Returns:
            dict or str: The book details if found, or a message indicating that the book was not found.

DATA
    library_db = {}

FILE
    c:\users\admin\downloads\ai assistence\library.py
-- More --
```



Task 8– Fibonacci Generator

Write a program to generate Fibonacci series up to n.

The initial code has:

- Global variables.
- Inefficient loop.
- No functions or modularity.

Task for Students:

- Refactor into a clean reusable function (generate_fibonacci).
- Add docstrings and test cases.

• Compare AI-refactored vs original.

Bad Code Version:

```
# fibonacci bad version
n=int(input("Enter limit: "))
a=0
b=1
print(a)
print(b)
for i in range(2,n):
c=a+b
print(c)
a=b
b=c
```

Code:

```python
#refactor the above code using functio-generate_fibnocci  with docstrings and generate a well documented code
def generate_fibonacci(n):
    """Generate Fibonacci sequence up to n terms.

    Args:
        n (int): The number of terms in the Fibonacci sequence to generate.

    Returns:
        list: A list containing the Fibonacci sequence up to n terms.
    """
    sequence = []
    a, b = 0, 1
    for i in range(n):
        sequence.append(a)
        a, b = b, a + b
    return sequence
# Example usage
if __name__ == "__main__":
    n = int(input("Enter limit: "))
    fibonacci_sequence = generate_fibonacci(n)
    print("Fibonacci Sequence:", fibonacci_sequence)
```

Output:

```
Enter limit: 5
Fibonacci Sequence: [0, 1, 1, 2, 3]
```

Task 9 – Twin Primes Checker

Twin primes are pairs of primes that differ by 2 (e.g., 11 and 13, 17 and 19).

The initial code has:

- Inefficient prime checking.
- No functions.
- Hardcoded inputs.

Task for Students:

- Refactor into is_prime(n) and is_twin_prime(p1, p2).
- Add docstrings and optimize.

• Generate a list of twin primes in a given range using AI.

Bad Code Version:

```
# twin primes bad version
a=11
b=13
fa=0
for i in range(2,a):
if a%i==0:
fa=1
fb=0
for i in range(2,b):
if b%i==0:
fb=1
if fa==0 and fb==0 and abs(a-b)==2:
print("Twin Primes")
else:
print("Not Twin Primes")
```

Code:

```python
#refactor the above code using into is_prime(n) and is_twin_prime(p1,p2) with docstrings and generate a well documented code
def is_prime(n):
    """Check if a number is prime.

    Args:
        n (int): The number to check for primality.
    Returns:
        bool: True if the number is prime, False otherwise.

    """
    if n <= 1:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True
#example usage
if __name__ == "__main__":
    a = 11
    b = 13

    if is_prime(a) and is_prime(b) and abs(a - b) == 2:
        print("Twin Primes")
    else:
        print("Not Twin Primes")
```

Output:

```
PS C:\Users\Admin\AppData\Local\Programs\Microsoft VS Code> 15
15
```

Task 10 – Refactoring the Chinese Zodiac Program

Objective

Refactor the given poorly structured Python script into a clean, modular, and

reusable implementation.

The current program reads a year from the user and prints the corresponding

Chinese Zodiac sign. However, the implementation contains repetitive

conditional logic, lacks modular design, and does not follow clean coding

principles.

Your task is to refactor the code to improve readability, maintainability, and

structure.

Chinese Zodiac Cycle (Repeats Every 12 Years)

1. Rat

2. Ox

3. Tiger

4. Rabbit

5. Dragon

6. Snake

7. Horse

8. Goat (Sheep)

9. Monkey

10. Rooster

11. Dog

12. Pig

```
# Chinese Zodiac Program (Unstructured Version)

# This code needs refactoring.

year = int(input("Enter a year: "))

if year % 12 == 0:
```

```
print("Monkey")
elif year % 12 == 1:
print("Rooster")
elif year % 12 == 2:
print("Dog")
elif year % 12 == 3:
print("Pig")
elif year % 12 == 4:
print("Rat")
elif year % 12 == 5:
print("Ox")
elif year % 12 == 6:
print("Tiger")
elif year % 12 == 7:
print("Rabbit")
elif year % 12 == 8:
print("Dragon")
elif year % 12 == 9:
print("Snake")
elif year % 12 == 10:
```

```
print("Horse")
elif year % 12 == 11:
print("Goat")
```

You must:

1. Create a reusable function: get_zodiac(year)

2. Replace the if-elif chain with a cleaner structure (e.g., list or dictionary).

3. Add proper docstrings.

4. Separate input handling from logic.

5. Improve readability and maintainability.

6. Ensure output remains correct.

Code:

Task 19 (Lucas Number Sequence – Test Case Design)

• Function: Generate Lucas sequence up to n terms.

(Starts with 2,1, then Fn = Fn-1 + Fn-2)

• Test Cases to Design:

• Normal: 5 → [2, 1, 3, 4, 7]

• Edge: 1 → [2]

• Negative: -5 → Error

• Large: 10 (last element = 76).

• Requirement: Validate correctness with pytest.

Code:

```python
# generate lucas sequence up to n terms and add pytest
#and test case to design normal function to generate lucas sequence and
# edge case for n=1 and negative and large values give last element of the sequence
def lucas_sequence(n):
    """
    Generate the Lucas sequence up to n terms.

    Args:
        n (int): The number of terms in the Lucas sequence to generate.

    Returns:
        list: A list containing the Lucas sequence up to n terms.
    """
    if n <= 0:
        return ["error"]
    elif n == 1:
        return [2]
    elif n == 2:
        return [2, 1]

    sequence = [2, 1]
    for i in range(2, n):
        next_term = sequence[i-1] + sequence[i-2]
        sequence.append(next_term)

    return sequence
# Test cases
print(lucas_sequence(5))  # Output: [2, 1, 3, 4, 7]
print(edge_case := lucas_sequence(1))  # Output: [2]
print(negative_case := lucas_sequence(-5))  # Output: [error]
print(large_value_case := lucas_sequence(10))  # Output: [2, 1, 3, 4, 7, 11, 18, 29, 47, 76]
```

Output:

```
[2, 1, 3, 4, 7]
[2]
['error']
[2]
['error']
['error']
[2, 1, 3, 4, 7, 11, 18, 29, 47, 76]
```

Task 20 (Vowel & Consonant Counter – Test Case Design)

• Function: Count vowels and consonants in string.

• Test Cases to Design:

• Normal: "hello" → (2,3)

• Edge: "" → (0,0)

• Only vowels: "aeiou" → (5,0)

Large: Long text

• Requirement: Validate correctness with pytest.

Code:

```python
#generate a code for vowels and consonants in string
#and provide test cases for the code and add pytest and test case to design
#normal function to count vowels and consonants in a string,edge case and only vowels and give length of the string
def count_vowels_consonants(input_string):
    """
    Count the number of vowels and consonants in a given string.

    Args:
        input_string (str): The string to be analyzed.
    Returns:
        tuple: A tuple containing the count of vowels and consonants.
    """
    vowels = 'aeiouAEIOU'
    vowel_count = 0
    consonant_count = 0
    for char in input_string:
        if char.isalpha():
            if char in vowels:
                vowel_count += 1
            else:
                consonant_count += 1
    return vowel_count, consonant_count
# Test cases
print(count_vowels_consonants("Hello"))  # Output: (2, 3)
print(edge_case := count_vowels_consonants(""))  # Output: (0, 0)
print(only_vowels := count_vowels_consonants("aeiou")) # Output: (5, 0)
```

Output:

```
(2, 3)
(0, 0)
(5, 0)
```